CLOUD COMPUTING · PROYECTO FINAL

# Análisis inteligente de imágenes en AWS

Amazon S3, Amazon Rekognition y Amazon RDS, con redundancia real detrás de un ALB y HAProxy

Universidad del Valle de Guatemala · Campus Altiplano
Docente: M.A. Ing. Dennys Tezén

11 de noviembre de 2026

EQUIPO

# Cada integrante administra un componente

| Integrante | Carné | Componente a cargo |
|---|---|---|
| Cristina Marie Jimenez Garcia | 20680 | Red, grupos de seguridad y ALB |
| Isidro Alexander Chuj Quijivix | 231621 | HAProxy (2 nodos) y health checks |
| Jorge José Adrián Monterroso Flores | 18610 | Aplicación, S3 y Rekognition |
| Edwin Manuel Valenzuela Chaj | 231715 | RDS, respaldos y recuperación |
| Kris Gregorio Garcia Xep | 23259 | Bastión, CloudFormation y documentación |

PROBLEMA Y OBJETIVO

# Clasificar imágenes a mano no escala, y un solo servidor es un punto de falla

## El problema

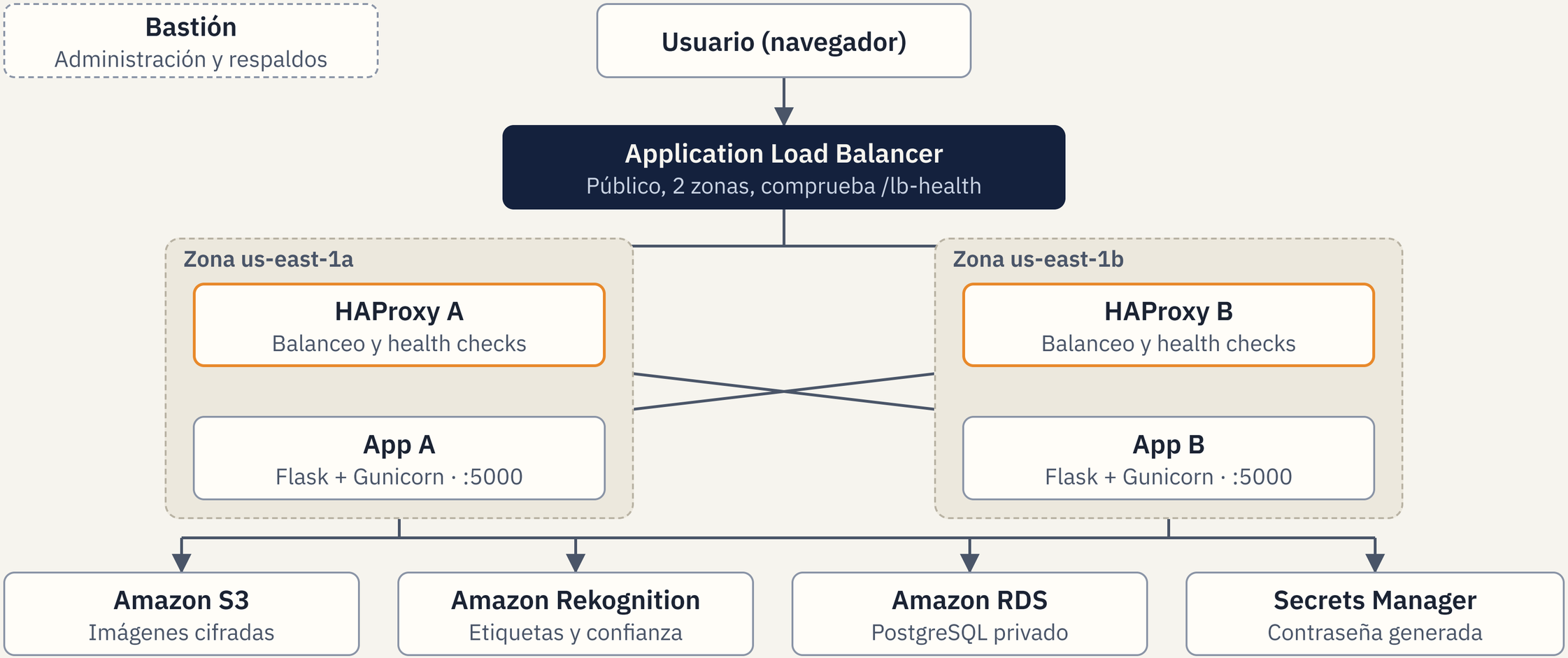
Etiquetar imágenes manualmente es lento y subjetivo. Una aplicación en un solo servidor deja de responder cuando ese servidor falla.

## Nuestro objetivo

Un sistema web en AWS que sube imágenes, las analiza con Rekognition, guarda el resultado y sigue disponible si falla una réplica.

ARQUITECTURA

# Ninguna capa de cómputo depende de un solo servidor



FLUJO DE UNA IMAGEN

# De la carga al resultado en cinco pasos

1

### Validar

JPG o PNG reales, por su firma, de hasta 5 MB

2

### Guardar

Se sube a S3 con un nombre UUID único

3

### Analizar

Rekognition DetectLabels, confianza mínima 70 %

4

### Registrar

Imagen, etiquetas, confianza y fecha en RDS

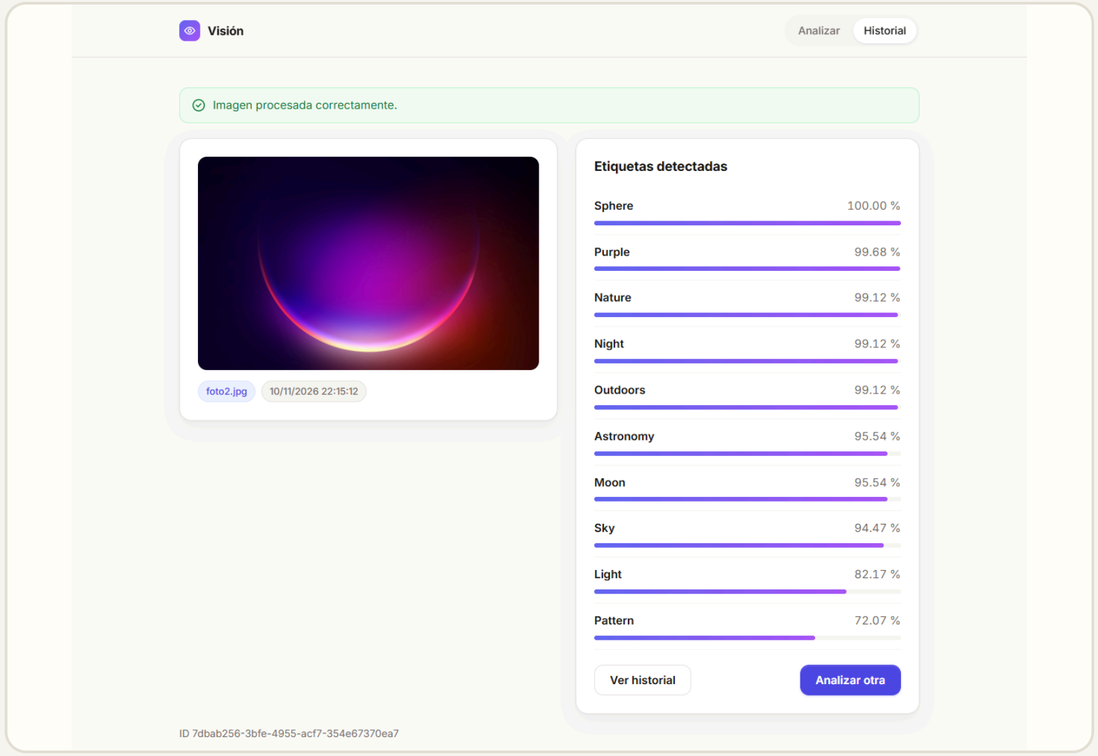
5

### Mostrar

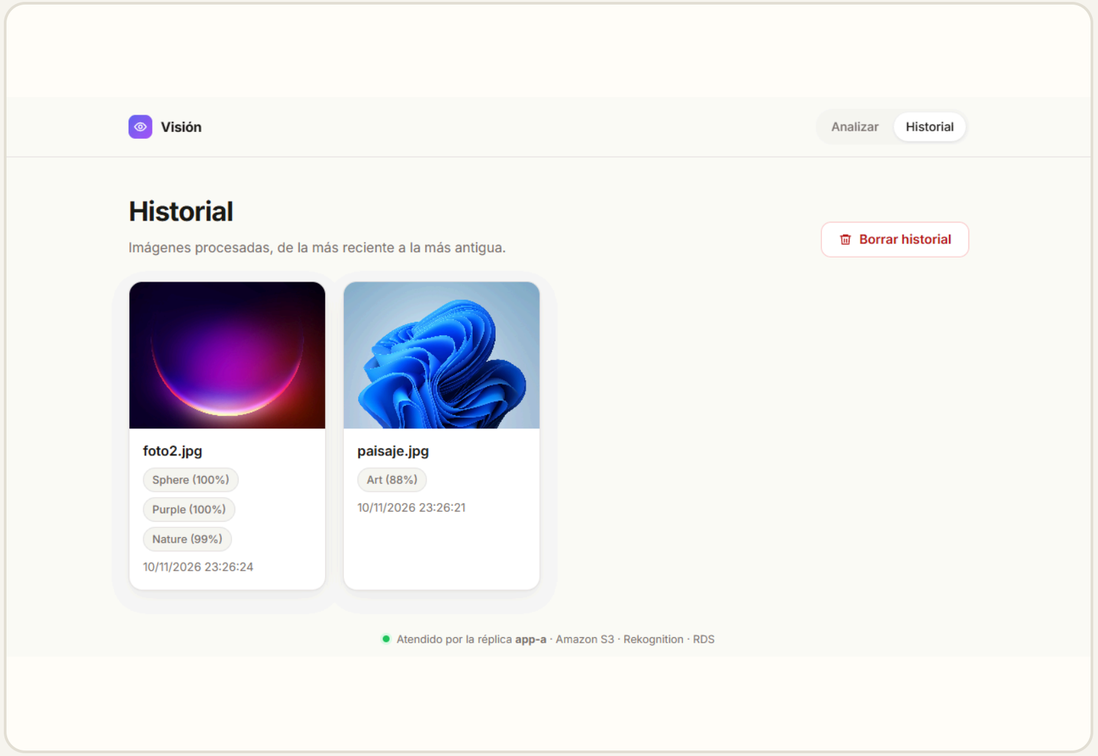
Resultado y entrada en el historial

LA APLICACIÓN

# Resultados con confianza e historial que se puede borrar




Resultado: imagen desde S3 y etiquetas de Rekognition




Historial: galería desde RDS y botón «Borrar historial»

TECNOLOGÍAS

# Servicios administrados donde se puede, código abierto donde aporta

| Capa | Tecnología | Por qué |
|---|---|---|
| Aplicación | Python, Flask, Gunicorn | Simple, con Boto3 como SDK oficial de AWS |
| Distribución | ALB + HAProxy | HAProxy obligatorio; ALB da entrada multizona |
| Objetos | Amazon S3 | Almacenamiento durable, privado y versionado |
| Análisis | Amazon Rekognition | Etiquetas sin entrenar modelos propios |
| Datos | Amazon RDS PostgreSQL | Historial con SQL, respaldos automáticos |
| Secretos | AWS Secrets Manager | Contraseña generada, fuera del código |
| Infraestructura | AWS CloudFormation | Todo se crea con una plantilla |

ALTA DISPONIBILIDAD

# Dos niveles de balanceo detectan y aíslan cada falla

## ALB → HAProxy

Consulta /lb-health cada 5 s. Tras 2 fallos retira el nodo HAProxy caído y envía todo al otro.

## HAProxy → App

Consulta /health cada 5 s. Si una conexión falla, reintenta en la otra réplica: el usuario no ve errores.

## Recuperación

systemd reinicia la app si cae y las actualizaciones usan recarga suave, sin cortes.

SEGURIDAD

# Cuatro controles básicos, aplicados en cada capa

### Privilegio mínimo

El rol solo permite S3 en uploads/ y backups/, DetectLabels y leer un secreto.

### Secretos fuera del código

Secrets Manager genera la contraseña de RDS; nadie la escribe ni la ve.

### Base de datos privada

RDS sin IP pública; grupos encadenados ALB → HAProxy → App → RDS.

### Validación de archivos

Tipo real por firma, máximo 5 MB, nombres UUID y HTML escapado.

INFRAESTRUCTURA COMO CÓDIGO

# Una plantilla de CloudFormation crea todo el sistema

**39**

recursos de AWS en un solo archivo

- VPC, 6 subredes, NAT Gateway y endpoint de S3
- 5 servidores EC2 que se configuran solos al arrancar
- ALB, target group, bucket, RDS y secreto
- Rol de IAM y 5 grupos de seguridad
- Un servidor perdido se recrea con Update stack

DEMOSTRACIÓN EN VIVO

# Apagamos servidores mientras el sistema sigue respondiendo

1. Subir una imagen y ver sus etiquetas y su confianza
2. Mostrar el objeto en S3 y el historial
3. Rechazo de un PDF renombrado y de una imagen de más de 5 MB
4. Ver cómo las respuestas alternan entre app-a y app-b
5. Detener app-a: HAProxy la marca DOWN y todo sigue en 200
6. Detener haproxy-a: el ALB lo retira y el sitio sigue arriba
7. Borrar el historial con confirmación y volver a analizar una imagen

RESULTADOS

# Las 14 pruebas cumplieron en el despliegue real

**0**

**errores al detener app-a**

HAProxy la marcó DOWN y reintentó en app-b

**~9 s**

**para retirar haproxy-a**

Apagado abrupto: 5 respuestas 502 de 41; retiro ordenado: 0 de 66

**0 / 133**

**errores al actualizar la app**

Con recarga suave de Gunicorn; reiniciando hubo 1 de 196

También cumplieron: etiquetas de Rekognition, validación de tipo y tamaño, historial y su borrado, SQL, RDS privado y código sin secretos.

RECUPERACIÓN Y RESPALDOS

# Cada componente tiene un camino de vuelta documentado

| Componente | Cómo se recupera |
|---|---|
| Réplica de la app | systemd la reinicia; si se pierde la instancia, Update stack la recrea |
| Nodo HAProxy | Validar con haproxy -c y reiniciar; o recrearlo con Update stack |
| Base de datos | Restauración a un punto en el tiempo (7 días) y snapshot manual |
| Datos lógicos | pg_dump diario desde el bastión hacia S3, en backups/ |
| Imágenes | Versionado de S3: se recupera la versión anterior |
| Actualizaciones | Una réplica a la vez con systemctl reload: 0 errores en 133 solicitudes |

CONCLUSIONES Y SIGUIENTES PASOS

# El sistema cumple los objetivos y sabemos qué mejorar

## Logramos

- Análisis automático con S3, Rekognition y RDS
- Sin punto único de falla en el cómputo
- Controles de seguridad en cada capa
- Infraestructura reproducible en una plantilla

## Mejoraríamos

- RDS Multi-AZ y un segundo NAT Gateway
- Auto Scaling Group en lugar de instancias fijas
- HTTPS con Certificate Manager
- Usuarios con Amazon Cognito

# ¿Preguntas?

Gracias por su atención